

This API architecture design for the Banking Management System adheres to RESTful principles, security best practices (OWASP/PCI-DSS), and the defined functional requirements.

1. API Overview

- **Base URL:** `https://api.bank-system.com/v1`
- **Format:** JSON (UTF-8)
- **Versioning:** URI-based (`/v1/`)
- **Concurrency:** Optimistic locking via `ETag` headers for balance updates.

2. Authentication & Security

- **Mechanism:** OAuth 2.0 + OpenID Connect (OIDC).
- **MFA:** A `X-MFA-Token` header is required for all state-changing operations (transfers > \$0, loan applications).
- **Rate Limiting:** Sliding window algorithm (e.g., 100 requests/min per user).
- **Encryption:** TLS 1.3 enforced.

3. Core Endpoints

Accounts

- `GET /accounts` – List all accounts for authenticated user.
- `GET /accounts/{id}` – Retrieve account balance and details.

Transactions

- `POST /transactions` – Execute an internal transfer.
- `GET /accounts/{id}/transactions` – Retrieve transaction history (pagination enabled).

Loan Management

- `POST /loans` – Submit a new loan application.
- `GET /loans` – (Employees) List pending loan applications.

- `PATCH /loans/{id}` – (Employees) Approve or Reject a loan.

Insights (AI)

- `GET /accounts/{id}/insights` – Get monthly spending breakdown.

4. Example Request & Response

POST /transactions (Transfer Funds)

Request Body:

```
{
  "source_account_id": "uuid-123",
  "dest_account_id": "uuid-456",
  "amount": 5500.00,
  "currency": "USD",
  "description": "Monthly rent"
}
```

Response Body (202 Accepted):

```
{
  "tx_id": "tx-uuid-999",
  "status": "PENDING_MFA",
  "message": "Transaction requires secondary authentication.",
  "links": {
    "verify": "/transactions/tx-uuid-999/verify"
  }
}
```

5. Status Codes

Code	Meaning	Usage
:---	:---	:---
`200 OK`	Success	General fetch operations.
`201 Created`	Created	Resource successfully created (Loan App).
`202 Accepted`	Accepted	Request processing (async or waiting for MFA).
`400 Bad Request`	Client Error	Validation failure (e.g., insufficient funds).
`401 Unauthorized`	Auth Error	Missing or expired token.

| `403 Forbidden` | Permission Error | MFA missing or insufficient role (e.g., Customer accessing Admin view). |

| `429 Too Many` | Throttling | API rate limit exceeded. |

6. Validation Logic

- **Request Schema Validation:** Middleware to enforce data types (e.g., `amount` must be `decimal`, `uuid` format for IDs).
- **Business Validation:**
 - * `POST /transactions`: Check if `source\_account.balance >= amount`.
 - * `PATCH /loans/{id}`: Verify the `processed\_by` field is updated with the employee's ID.
- **Idempotency:** Implement `Idempotency-Key` header for `POST /transactions` to prevent duplicate charges during network retries.

7. Error Responses

Standardized error body to aid debugging while hiding system internals:

```
{
  "error": {
    "code": "INSUFFICIENT_FUNDS",
    "message": "The account balance is too low to complete this transfer.",
    "request_id": "req-xyz-123",
    "timestamp": "2023-10-27T10:00:00Z"
  }
}
```

8. Architectural Considerations

- **Audit Logging:** Every sensitive request (any `PATCH`, `POST`, `DELETE`) is intercepted by an Aspect-Oriented Programming (AOP) layer that captures user intent, timestamp, and IP address for the `AuditLogs` table.
- **Database Isolation:** Use Read-Replicas for `GET` requests (`/insights`, `/accounts`) to maintain performance on the primary transactional database during high load.

- **Compliance:** All sensitive endpoints (Transactions, Loans) pass through a Data Privacy Layer that masks Personally Identifiable Information (PII) before logging/auditing.